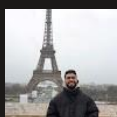




Encrypt Data with AWS KMS



Hayder Ben Abdelhamid

⊗ **Access denied**

You don't have permission to *kms:Decrypt*. To request access, copy the following text and send it to your AWS administrator. [Learn more about troubleshooting access denied errors.](#) 

User: arn:aws:iam::471112976395:user/nextwork-kms-user

Action: kms:Decrypt

On resource(s): arn:aws:kms:us-east-2:471112976395:key/f06272c5-495f-4b77-930c-f45308ac3730

Context: no identity-based policy allows the action





Introducing Today's Project!

In this project, I will demonstrate using encryption to secure data. The goal is to create encryption keys with AWS KMS(Key Management Service), encrypt a DynamoDB table's data with that key, then test access using IAM users.

Tools and concepts

Services I used include AWS KMS (Key Management Service), DynamoDB, AWS IAM. Key concepts I learnt include encryption, database tables, KMS using permission to actions rather than just access to the key itself and creating a user to test access.

Project reflection

This project took me approximately 1 hour. The most challenging part was understanding how encryption works differently from other access controls. It was most rewarding to see the test user get access to encryption.

I chose to do this project today to learn all about encryption, securing data and how it actually works.

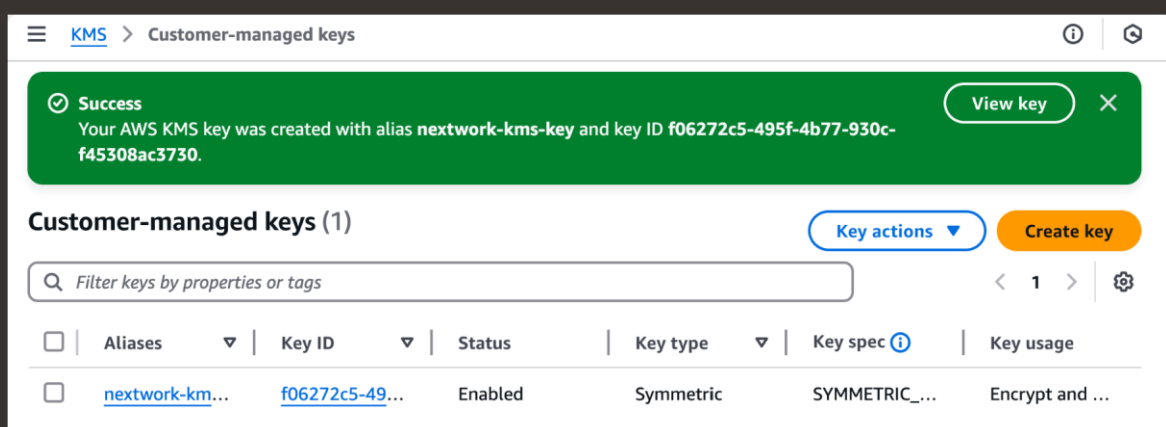


Encryption and KMS

Encryption is the process of turning original/plaintext data into a secure format. Companies and developers do this to secure their data from unauthorized users. Encryption keys are the secure code that informs an algorithm on how should encrypt.

AWS KMS is a vault for our encryption keys. Key management systems are important because they help us secure and manage the keys we use to encrypt data . Unauthorized access to the key = exposing our data, which puts our security at risk.

Encryption keys are broadly categorized as symmetric and asymmetric. I set up a symmetric key because i will be using the exact same key to encrypt and decrypt my data. Asymmetric keys would be a good choice if we need different keys for de/encryption .





Encrypting Data

My encryption key will safeguard data in DynamoDB, which is a fast and flexible AWS database service. DynamoDB is great for applications that need fast access to large amounts of data e.g. Gaming.

The different encryption options in DynamoDB include DynamoDB-owned, AWS managed and customer managed. Their differences are based on who creates and manages the key and whether we have visibility. I selected customer managed key option to use our created KMS key

Encryption at rest

Encryption key management

- ☐ **Owned by Amazon DynamoDB**
The AWS KMS key is owned and managed by DynamoDB. You are not charged an additional fee for using this key.
- ☐ **AWS managed key**
Key alias: aws/dynamodb. The key is stored in your account and is managed by AWS Key Management Service (AWS KMS). AWS KMS charges apply.
- ☒ **Stored in your account, and owned and managed by you**
The key is stored in your account and is owned and managed by you. AWS KMS charges apply.

 `arn:aws:kms:us-west-1:471112976395:key/f2df772c-c` 



Data Visibility

Rather than controlling who has access to the key, KMS manages user permissions by controlling who has access to the key manager user permissions by controlling the actions that people can do with that key. In our case, even if we gave our test user the permission to see the key, it would need permission to decrypt.

Despite encrypting my DynamoDB table, I could still see the table's items because i'm the user of the key. DynamoDB uses transparent data encryption, which means it does the encryption/decryption process for us because it knows we're authorized.

nextwork-kms-table Autopreview View table details

▶ Scan or query items
Expand to query or scan items.

✔ Completed. Read capacity units consumed: 2

Items returned (1) Refresh Actions Create item

☐

id (String)

▼

☐

[1](#)



Denying Access

I configured a new IAM user to validate whether unauthorized users can access encrypted data. The permission policies I granted this user are DynamoDB full access, but Not encryption/decryption permission with AWS KMS.

After accessing the DynamoDB table as the test user, I encountered an access denied error message because our test user has no access to decryption. This confirmed that encryption keys can be used to secure data .

Access denied

You don't have permission to *kms:Decrypt*. To request access, copy the following text and send it to your AWS administrator. [Learn more about troubleshooting access denied errors.](#) 

User: arn:aws:iam::471112976395:user/nextwork-kms-user

Action: kms:Decrypt

On resource(s): arn:aws:kms:us-east-2:471112976395:key/f06272c5-495f-4b77-930c-f45308ac3730

Context: no identity-based policy allows the action





EXTRA: Granting Access

To let my test user use the encryption key, I made it a key user in the KMS console! My key's policy was updated to allow the nextwork-kms-user to encrypt, decrypt, re-encrypt using the key.

Using the test user, I retried accessing the DynamoDB table. I observed that the user can see the data inside again, which confirmed that making it a key user is an effective way to authorize someone to see encrypted data.

Encryption secures data instead of an entire resource or service. I could combine encryption with other access control tools like security groups and permission policies to have two layers of security the resource level, and then the data level.

```
{
  "Sid": "Allow use of the key",
  "Effect": "Allow",
  "Principal": {
    "AWS": [
      "arn:aws:iam::471112976395:user/Nat-IAM-Admin",
      "arn:aws:iam::471112976395:user/nextwork-kms-user"
    ]
  },
  "Action": [
    "kms:Encrypt",
    "kms:Decrypt",
    "kms:ReEncrypt*",
    "kms:GenerateDataKey*",
    "kms:DescribeKey"
  ],
  "Resource": "*"
},
```

